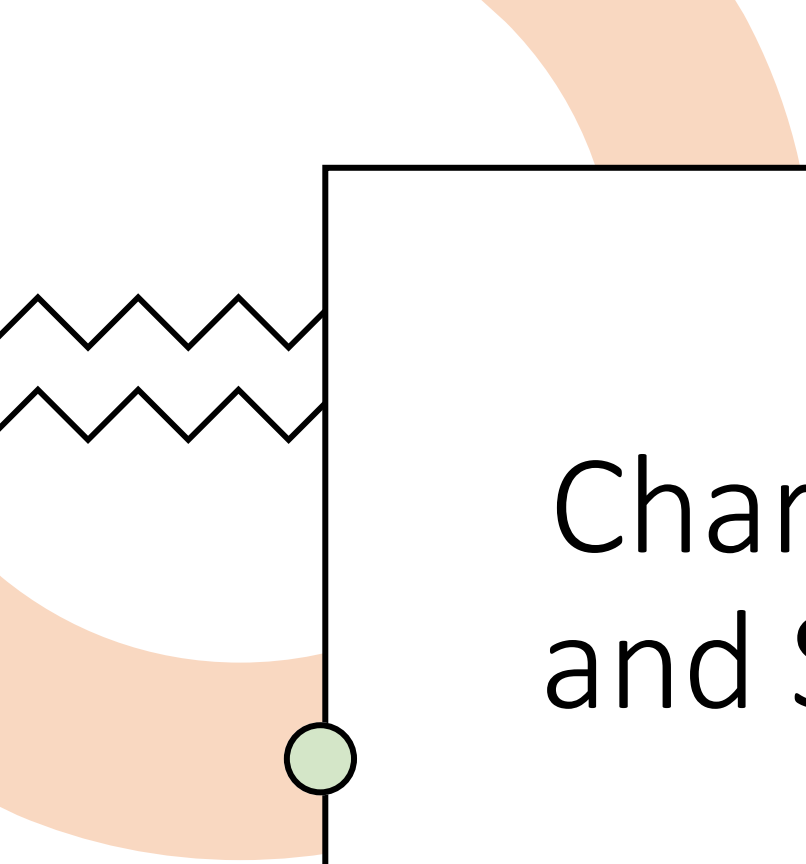




Characters and Strings



We will learn

- Fundamentals of strings and characters
- Character-handling library
- Standard input/output library functions
- String-manipulation functions of the string-handling library
 - Manipulation functions of the string-handling library
 - Comparison functions of the string-handling library
 - Other functions of the string-handling library

Motivation

- Introduce some standard library functions
 - Easy string and character processing
 - Programs can process characters, strings, lines of text, and blocks of memory
- These techniques used to make
 - Word processors
 - Page layout software
 - Typesetting programs

Fundamentals of Strings and Characters

- Characters

- Character constant

- An `int` value represented as a character in single quotes
 - `'z'` represents the integer value of `z`

- Strings

- Series of characters treated as a single unit
 - Can include letters, digits and special characters (`*`, `/`, `$`)
 - String literal (string constant) - written in double quotes
 - `"Hello"`
 - Strings are arrays of characters
 - Access a string via a *pointer* to its first character
 - Value of string is the address of first character

Fundamentals of Strings and Characters

- String definitions/Initializations

- Define as a character array or a variable of type `char *`

- ```
char color[] = "blue";
```

- ```
char *colorPtr = "blue";
```

- Remember that strings represented as character arrays end with `'\0'`

- `color` has 5 elements

- Inputting strings

- Use `scanf`

- ```
scanf("%s", word);
```

- Copies input into `word[]`

- Do not need `&` (because a string is a pointer)

- How to read only required no. of characters?

- ```
scanf("%19s", word);
```

- Remember to leave room in the array for `'\0'`: otherwise overwrite/error

Handling strings

- `char word[20];`
 - `scanf("%19s", word);`
 - `scanf("%20s", word);`
 - `scanf("%s", word);`
- `// Supercalifragilisticexpialidocious`
- `// New Delhi`

Character Handling Library

- Character handling library
 - Includes functions to perform useful tests and manipulations of character data
 - Each function receives a character (an `int`) or EOF as an argument
- The following slide contains a table of all the functions in `<ctype.h>`

Character-
handling
library
functions

character-handling library (<ctype.h>)

Example: Character Handling

```
1  /* Fig. 8.2: fig08_02.c
2     Using functions isdigit, isalpha, isalnum, and isxdigit */
3  #include <stdio.h>
4  #include <ctype.h>
5
6  int main( void )
7  {
8      printf( "%s\n%s%s\n%s%s\n\n", "According to isdigit: ",
9          isdigit( '8' ) ? "8 is a " : "8 is not a ", "digit",
10         isdigit( '#' ) ? "# is a " : "# is not a ", "digit" );
11
12     printf( "%s\n%s%s\n%s%s\n%s%s\n\n",
13         "According to isalpha:",
14         isalpha( 'A' ) ? "A is a " : "A is not a ", "letter",
15         isalpha( 'b' ) ? "b is a " : "b is not a ", "letter",
16         isalpha( '&' ) ? "& is a " : "& is not a ", "letter",
17         isalpha( '4' ) ? "4 is a " : "4 is not a ", "letter" );
18
```

isdigit tests if a character is a decimal digit

isalpha tests if a character is a letter

Example: Character Handling

```
19  printf( "%s\n%s%s\n%s%s\n%s%s\n\n",
20          "According to isalnum:",
21          isalnum( 'A' ) ? "A is a " : "A is not a ",
22          "digit or a letter",
23          isalnum( '8' ) ? "8 is a " : "8 is not a ",
24          "digit or a letter",
25          isalnum( '#' ) ? "# is a " : "# is not a ",
26          "digit or a letter" );
27
28  printf( "%s\n%s%s\n%s%s\n%s%s\n%s%s\n",
29          "According to isxdigit:",
30          isxdigit( 'F' ) ? "F is a " : "F is not a ",
31          "hexadecimal digit",
32          isxdigit( 'J' ) ? "J is a " : "J is not a ",
33          "hexadecimal digit",
34          isxdigit( '7' ) ? "7 is a " : "7 is not a ",
35          "hexadecimal digit",
36          isxdigit( '$' ) ? "$ is a " : "$ is not a ",
```

isdigit tests if a character is a decimal digit or a letter

isxdigit tests if a character is a hexadecimal digit

Example: Character Handling

```
37     "hexadecimal digit",  
38     isxdigit('f') ? "f is a " : "f is not a ",  
39     "hexadecimal digit" );  
40  
41     return 0; /* indicates successful termination */  
42  
43 } /* end main */
```

According to isdigit:

8 is a digit

is not a digit

According to isalpha:

A is a letter

b is a letter

& is not a letter

4 is not a letter

According to isalnum:

A is a digit or a letter

8 is a digit or a letter

is not a digit or a letter

According to isxdigit:

F is a hexadecimal digit

J is not a hexadecimal digit

7 is a hexadecimal digit

\$ is not a hexadecimal digit

f is a hexadecimal digit

Example: Character Handling

```
1  /* Fig. 8.3: fig08_03.c
2     Using functions islower, isupper, tolower, toupper */
3  #include <stdio.h>
4  #include <ctype.h>
5
6  int main( void )
7  {
8      printf( "%s\n%s\n%s\n%s\n%s\n",
9              "According to islower:",
10             islower( 'p' ) ? "p is a " : "p is not a ",
11             "lowercase letter",
12             islower( 'P' ) ? "P is a " : "P is not a ",
13             "lowercase letter",
14             islower( '5' ) ? "5 is a " : "5 is not a ",
15             "lowercase letter",
16             islower( '!' ) ? "! is a " : "! is not a ",
17             "lowercase letter" );
18
19      printf( "%s\n%s\n%s\n%s\n%s\n",
20              "According to isupper:",
21             isupper( 'D' ) ? "D is an " : "D is not an ",
22             "uppercase letter",
23             isupper( 'd' ) ? "d is an " : "d is not an ",
24             "uppercase letter",
25             isupper( '8' ) ? "8 is an " : "8 is not an ",
26             "uppercase letter",
27             isupper( '$' ) ? "$ is an " : "$ is not an ",
28             "uppercase letter" );
29  }
```

islower tests if a character is a lowercase letter

isupper tests if a character is an uppercase letter

Example: Character Handling

```
30  printf( "%s%c\n%s%c\n%s%c\n%s%c\n",
31          "u converted to uppercase is ", toupper( 'u' ),
32          "7 converted to uppercase is ", toupper( '7' ),
33          "$ converted to uppercase is ", toupper( '$' ),
34          "L converted to lowercase is ", tolower( 'L' ) );
35
36  return 0; /* indicates successful termination */
37
38 } /* end main */
```

toupper and **tolower** convert
letters to upper or lower case

According to islower:

p is a lowercase letter
P is not a lowercase letter
5 is not a lowercase letter
! is not a lowercase letter

According to isupper:

D is an uppercase letter
d is not an uppercase letter
8 is not an uppercase letter
\$ is not an uppercase letter

u converted to uppercase is U
7 converted to uppercase is 7
\$ converted to uppercase is \$
L converted to lowercase is l

Standard Input/Output Library Functions

- Functions in <stdio.h>
- Used to manipulate character and string data

```
int c;  
while ((c = getchar()) != '\n' && c != EOF) {  
    putchar(c); // echoes the input  
}
```

Function prototype	Function description
<code>int getchar(void);</code>	Inputs the next character from the standard input and returns it as an integer.
<code>char *gets(char *s);</code>	Inputs characters from the standard input into the array <i>S</i> until a newline or end-of-file character is encountered. A terminating null character is appended to the array. Returns the string inputted into <i>S</i> . Note that an error will occur if <i>S</i> is not large enough to hold the string.
<code>int putchar(int c);</code>	Prints the character stored in <i>C</i> and returns it as an integer.
<code>int puts(const char *s);</code>	Prints the string <i>S</i> followed by a newline character. Returns a non-zero integer if successful, or EOF if an error occurs.
<code>int sprintf(char *s, const char *format, ...);</code>	Equivalent to <code>printf</code> , except the output is stored in the array <i>S</i> instead of printed on the screen. Returns the number of characters written to <i>S</i> , or EOF if an error occurs.
<code>int sscanf(char *s, const char *format, ...);</code>	Equivalent to <code>scanf</code> , except the input is read from the array <i>S</i> rather than from the keyboard. Returns the number of items successfully read by the function, or EOF if an error occurs.

Example: String I/O

```
1  /* Fig. 8.13: fig08_13.c
2     Using gets and putchar */
3  #include <stdio.h>
4
5  void reverse( const char * const sPtr ); /* prototype */
6
7  int main( void )
8  {
9     char sentence[ 80 ]; /* create char array */
10
11    printf( "Enter a line of text:\n" );
12
13    /* use gets to read line of text */
14    gets( sentence );
15
16    printf( "\nThe line printed backward is:\n" );
17    reverse( sentence );
18
19    return 0; /* indicates successful termination */
20
21 } /* end main */
```

← gets reads a line of text from the user

Example: String I/O

```
22
23 /* recursively outputs characters in string in reverse order */
24 void reverse( const char * const sPtr )
25 {
26     /* if end of the string */
27     if ( sPtr[ 0 ] == '\0' ) { /* base case */
28         return;
29     } /* end if */
30     else { /* if not end of the string */
31         reverse( &sPtr[ 1 ] ); /* recursion step */
32
33         putchar( sPtr[ 0 ] ); /* use putchar to display character */
34     } /* end else */
35
36 } /* end function reverse */
```

putchar prints a single character on the screen

Enter a line of text:
Characters and Strings

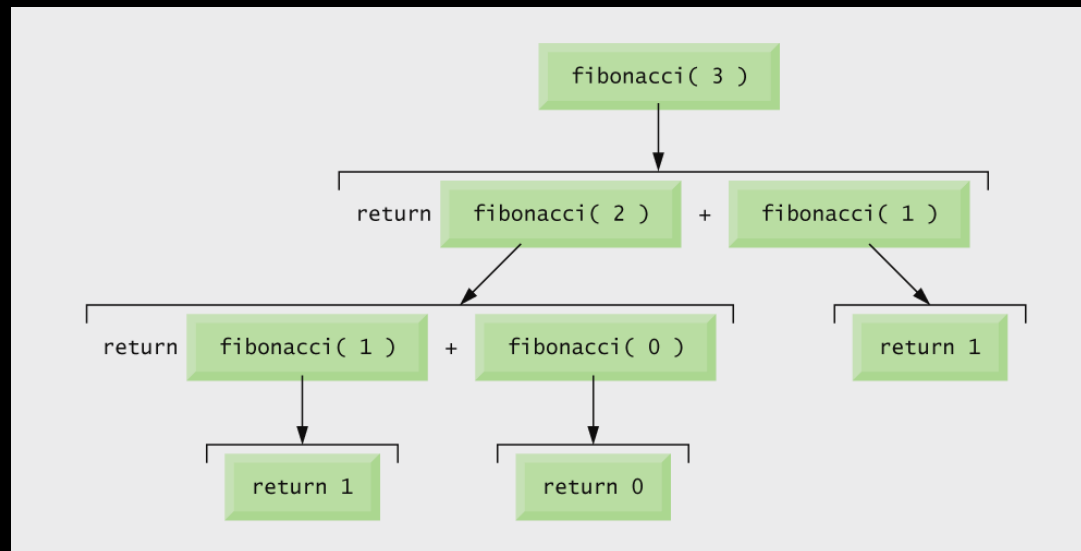
The line printed backward is:
sgnirts dna sretcarahC

Enter a line of text:
able was I ere I saw elba

The line printed backward is:
able was I ere I saw elba

Recall Recursive Functions

- The function calls itself in its body.
- Do itself again and again until reach the base case.



Detailed Analysis

- Step 1: reverse("abc")
 - Step 2: reverse(&sPtr[1]) → reverse("bc") (Do first)
 - putchar(sPtr[0]) → putchar('a') (Do next)
- Step 2: reverse("bc")
 - Step 3: reverse(&sPtr[1]) → reverse("c") (Do first)
 - putchar(sPtr[0]) → putchar('b') (Do next)
- Step 3: reverse("c")
 - Step 4: reverse(&sPtr[1]) → reverse("\0") (Do first)
 - putchar(sPtr[0]) → putchar('c') (Do next)
- Step 4: reverse("\0")
 - base case → return
- Step 4 → Step 3 → Step 2 → Step 1 "cba"

Example: String I/O

```
1  /* Fig. 8.14: fig08_14.c
2     Using getchar and puts */
3  #include <stdio.h>
4
5  int main( void )
6  {
7      char c;          /* variable to hold character input by user */
8      char sentence[ 80 ]; /* create char array */
9      int i = 0;       /* initialize counter i */
10
11     /* prompt user to enter line of text */
12     puts( "Enter a line of text:" );
13
14     /* use getchar to read each character */
15     while ( ( c = getchar() ) != '\n' ) {
16         sentence[ i++ ] = c;
17     } /* end while */
18
19     sentence[ i ] = '\0'; /* terminate string */
20
```

puts prints a line of text on the screen

getchar reads a single character from the user

Example: String I/O

```
21  /* use puts to display sentence */
22  puts( "\nThe line entered was:" );
23  puts( sentence );
24
25  return 0; /* indicates successful termination */
26
27 } /* end main */
```

Enter a line of text:
This is a test.

The line entered was:
This is a test.

Example: String I/O

```
1  /* Fig. 8.15: fig08_15.c
2     Using sprintf */
3  #include <stdio.h>
4
5  int main( void )
6  {
7     char s[ 80 ]; /* create char array */
8     int x;        /* x value to be input */
9     double y;     /* y value to be input */
10
11    printf( "Enter an integer and a double:\n" );
12    scanf( "%d%lf", &x, &y );
13
14    sprintf( s, "integer:%6d\ndouble:%8.2f", x, y );
15
16    printf( "%s\n%s\n",
17           "The formatted output stored in array s is:", s );
18
19    return 0; /* indicates successful termination */
20
21 } /* end main */
```

sprintf prints a line of text into an array
like **printf** prints text on the screen

```
Enter an integer and a double:
298 87.375
The formatted output stored in array s is:
integer:   298
double:   87.38
```

Example: String I/O

```
1  /* Fig. 8.16: fig08_16.c
2     Using sscanf */
3  #include <stdio.h>
4
5  int main( void )
6  {
7     char s[] = "31298 87.375"; /* initialize array s */
8     int x;    /* x value to be input */
9     double y; /* y value to be input */
10
11     sscanf( s, "%d%lf", &x, &y );
12
13     printf( "%s\n%s%6d\n%s%8.3f\n",
14            "The values stored in character array s are:",
15            "integer:", x, "double:", y );
16
17     return 0; /* indicates successful termination */
18
19 } /* end main */
```

← **sscanf** reads a line of text from an array like
scanf reads text from the user

The values stored in character array s are:
integer: 31298
double: 87.375

String Manipulation Functions of the String Handling Library

- String handling library has functions to
 - Manipulate string data
 - Search strings
 - Tokenize strings
 - Determine string length

Function prototype	Function description
<code>char *strcpy(char *s1, const char *s2)</code>	Copies string <code>s2</code> into array <code>s1</code> . The value of <code>s1</code> is returned.
<code>char *strncpy(char *s1, const char *s2, size_t n)</code>	Copies at most <code>n</code> characters of string <code>s2</code> into array <code>s1</code> . The value of <code>s1</code> is returned.
<code>char *strcat(char *s1, const char *s2)</code>	Appends string <code>s2</code> to array <code>s1</code> . The first character of <code>s2</code> overwrites the terminating null character of <code>s1</code> . The value of <code>s1</code> is returned.
<code>char *strncat(char *s1, const char *s2, size_t n)</code>	Appends at most <code>n</code> characters of string <code>s2</code> to array <code>s1</code> . The first character of <code>s2</code> overwrites the terminating null character of <code>s1</code> . The value of <code>s1</code> is returned.

Example: String Manipulation

```
1  /* Fig. 8.18: fig08_18.c
2     Using strcpy and strncpy */
3  #include <stdio.h>
4  #include <string.h>
5
6  int main( void )
7  {
8      char x[] = "Happy Birthday to You"; /* initialize char array x */
9      char y[ 25 ]; /* create char array y */
10     char z[ 15 ]; /* create char array z */
11
12     /* copy contents of x into y */
13     printf( "%s%s\n%s%s\n",
14         "The string in array x is: ", x,
15         "The string in array y is: ", strcpy( y, x ) );
16
17     /* copy first 14 characters of x into z. Does not copy null
18        character */
19     strncpy( z, x, 14 );
20
21     z[ 14 ] = '\0'; /* terminate string in z */
22     printf( "The string in array z is: %s\n", z );
23
24     return 0; /* indicates successful termination */
25
26 } /* end main */
```

strcpy copies string **x**
into character array **y**

strncpy copies 14 characters of
string **x** into character array **z**

Note that **strncpy** does not
automatically append a null character

Example: String Manipulation

```
1  /* Fig. 8.18: fig08_18.c
2     Using strcpy and strncpy */
3  #include <stdio.h>
4  #include <string.h>
5
6  int main( void )
7  {
8     char x[] = "Happy Birthday to You"; /* initialize char array x */
9     char y[ 25 ]; /* create char array y */
10    char z[ 15 ]; /* create char array z */
11
12    /* copy contents of x into y */
13    printf( "%s%s\n%s%s\n",
14           "The string in array x is: ", x,
15           "The string in array y is: ", strcpy( y, x ) );
16
17    /* copy first 14 characters of x into z. Does not copy null
18       character */
19    strncpy( z, x, 14 );
20
21    z[ 14 ] = '\0'; /* terminate string in z */
22    printf( "The string in array z is: %s\n", z );
23
24    return 0; /* indicates successful termination */
25
26 } /* end main */
```

strcpy copies string **x**
into character array **y**

strncpy copies 14 characters of
string **x** into character array **z**

Note that **strncpy** does not
automatically append a null character

```
The string in array x is: Happy Birthday to You
The string in array y is: Happy Birthday to You
The string in array z is: Happy Birthday
```

Example: String Manipulation

⚠ 2. `strncpy(dest, src, n)`

This one is trickier.

➤ **Case A: if `strlen(src) < n`**

→ Copies all characters and adds a null terminator.

➤ **Case B: if `strlen(src) >= n`**

→ Copies exactly `n` characters, but does NOT add '`\0`'.

So you must add it manually in this case.

Example: String Manipulation

```
1  /* Fig. 8.19: fig08_19.c
2     Using strcat and strncat */
3  #include <stdio.h>
4  #include <string.h>
5
6  int main( void )
7  {
8     char s1[ 20 ] = "Happy "; /* initialize char array s1 */
9     char s2[] = "New Year "; /* initialize char array s2 */
10    char s3[ 40 ] = "";      /* initialize char array s3 to empty */
11
12    printf( "s1 = %s\ns2 = %s\n", s1, s2 );
13
14    /* concatenate s2 to s1 */
15    printf( "strcat( s1, s2 ) = %s\n", strcat( s1, s2 ) );
16
17    /* concatenate first 6 characters of s1 to s3. Place '\0'
18       after last character */
19    printf( "strncat( s3, s1, 6 ) = %s\n", strncat( s3, s1, 6 ) );
20
```

strcat adds the characters of string **s2** to the end of string **s1**

strncat adds the first 6 characters of string **s1** to the end of string **s3**. A terminating null character is automatically appended to the result.

Example: String Manipulation

```
21  /* concatenate s1 to s3 */
22  printf( "strcat( s3, s1 ) = %s\n", strcat( s3, s1 ) );
23
24  return 0; /* indicates successful termination */
25
26 } /* end main */
```

Example: String Manipulation

```
21  /* concatenate s1 to s3 */
22  printf( "strcat( s3, s1 ) = %s\n", strcat( s3, s1 ) );
23
24  return 0; /* indicates successful termination */
25
26 } /* end main */
```

```
s1 = Happy
s2 = New Year
strcat( s1, s2 ) = Happy New Year
strncat( s3, s1, 6 ) = Happy
strcat( s3, s1 ) = Happy Happy New Year
```

Comparison Functions of the String Handling Library

- Comparing strings

- Computer compares numeric ASCII codes of characters in string
- Appendix D has a list of character codes

```
int strcmp( const char *s1, const char *s2 );
```

- Compares string s1 to s2
- Returns a negative number if $s1 < s2$, zero if $s1 == s2$ or a positive number if $s1 > s2$

```
int strncmp( const char *s1, const char *s2, size_t n );
```

- Compares up to n characters of string s1 to s2
- Returns values as above

Example: String Comparison

```
1  /* Fig. 8.21: fig08_21.c
2     Using strcmp and strncmp */
3  #include <stdio.h>
4  #include <string.h>
5
6  int main( void )
7  {
8     const char *s1 = "Happy New Year"; /* initialize char pointer */
9     const char *s2 = "Happy New Year"; /* initialize char pointer */
10    const char *s3 = "Happy Holidays"; /* initialize char pointer */
11
12    printf("%s%s\n%s%s\n%s%s\n\n%s%2d\n%s%2d\n%s%2d\n\n",
13          "s1 = ", s1, "s2 = ", s2, "s3 = ", s3,
14          "strcmp(s1, s2) = ", strcmp( s1, s2 ),
15          "strcmp(s1, s3) = ", strcmp( s1, s3 ),
16          "strcmp(s3, s1) = ", strcmp( s3, s1 ) );
17
```

strcmp compares
string **s1** to string **s2**

Example: String Comparison

```
18  printf("%s%2d\n%s%2d\n%s%2d\n",
19         "strncmp(s1, s3, 6) = ", strncmp( s1, s3, 6 ),
20         "strncmp(s1, s3, 7) = ", strncmp( s1, s3, 7 ),
21         "strncmp(s3, s1, 7) = ", strncmp( s3, s1, 7 ) );
22
23  return 0; /* indicates successful termination */
24
25 } /* end main */
```

strncmp compares the first 6 characters of string **s1** to the first 6 characters of string **s3**

Example: String Comparison

```
18  printf("%s%2d\n%s%2d\n%s%2d\n",
19          "strncmp(s1, s3, 6) = ", strncmp( s1, s3, 6 ),
20          "strncmp(s1, s3, 7) = ", strncmp( s1, s3, 7 ),
21          "strncmp(s3, s1, 7) = ", strncmp( s3, s1, 7 ) );
22
23  return 0; /* indicates successful termination */
24
25 } /* end main */
```

strncmp compares the first 6 characters of string **s1** to the first 6 characters of string **s3**

```
s1 = Happy New Year
s2 = Happy New Year
s3 = Happy Holidays
```

```
strcmp(s1, s2) = 0
strcmp(s1, s3) = 1
strcmp(s3, s1) = -1
```

```
strncmp(s1, s3, 6) = 0
strncmp(s1, s3, 7) = 1
strncmp(s3, s1, 7) = -1
```

Example: String Length

```
1  /* Fig. 8.38: fig08_38.c
2     Using strlen */
3  #include <stdio.h>
4  #include <string.h>
5
6  int main( void )
7  {
8     /* initialize 3 char pointers */
9     const char *string1 = "abcdefghijklmnopqrstuvwxy";
10    const char *string2 = "four";
11    const char *string3 = "Boston";
12
13    printf("%s\\%s\\%s%lu\\n%s\\%s\\%s%lu\\n%s\\%s\\%s%lu\\n",
14        "The length of ", string1, " is ",
15        ( unsigned long ) strlen( string1 ),
16        "The length of ", string2, " is ",
17        ( unsigned long ) strlen( string2 ),
18        "The length of ", string3, " is ",
19        ( unsigned long ) strlen( string3 ) );
20
21    return 0; /* indicates successful termination */
22
23 } /* end main */
```

```
size_t strlen( const char *s );
```

Determines the length of string *s*. The number of characters preceding the terminating null character is returned.

strlen returns the length of **string1**

```
The length of "abcdefghijklmnopqrstuvwxy" is 26
The length of "four" is 4
The length of "Boston" is 6
```